UNIVERSIDAD DE LAS AMÉRICAS

Facultad de Ingeniería y Ciencias Aplicadas
Ingeniería de Software

Dizeo
Planificación y asignación de personal en obras

ISWZ2102 · Programación III
Avance de proyecto · Progreso 1

Daniel Valarezo
Esteban Figueroa
Julián Jiménez
Cuarto integrante: por definir

Quito, Ecuador · 18 de octubre de 2026

Estado: propuesta técnica preliminar para revisión del equipo. La consigna requiere entre cuatro y cinco integrantes. La asignación de un cuarto integrante y la aprobación grupal del UML permanecen pendientes.

Base: documento de Ingeniería de Requerimientos aportado. Este avance conserva el dominio y sus RF-01 a RF-25; desarrolla la comparación algorítmica y el plan previo a implementación.

1. Formulación del problema

Dizeo es el caso de estudio de una empresa de construcción y acabados que atiende Quito y sus valles. El documento de requerimientos describe servicios de pintura, impermeabilización, gypsum, obra gris, señalización y cerramientos. Cada obra reúne actividades que necesitan personas con especialidades determinadas y fechas compatibles con los compromisos del cliente. Cuando varios frentes comparten trabajadores, una decisión local puede afectar la ejecución de otra obra. El problema se concentra en coordinar esa capacidad limitada, no en registrar únicamente nombres de clientes o trabajos (Valarezo et al., 2026).

La necesidad identificada consiste en contar con información consistente para decidir quién puede realizar una actividad, durante qué intervalo y después de cuáles trabajos previos. El caso plantea posibles dobles asignaciones, actividades sin responsable y cambios que no llegan oportunamente a los involucrados. Estas situaciones constituyen hipótesis de diagnóstico: todavía no existen entrevistas o mediciones que permitan afirmar su frecuencia en Dizeo. Durdyev y Hosseini (2020) identifican la comunicación deficiente, la falta de coordinación y la planificación inadecuada entre causas recurrentes de retrasos en construcción. Su revisión justifica investigar esas variables, pero no prueba que todas ocurran en esta empresa.

Los actores principales son la dirección, el supervisor y los trabajadores. La dirección necesita conocer los compromisos y la capacidad disponible. El supervisor organiza el plan semanal, valida especialidades y resuelve cambios diarios. Los trabajadores informan disponibilidad, ejecutan tareas y reportan avances o bloqueos. Los clientes son afectados por las fechas y reprogramaciones, aunque no se prevé que operen la primera versión. Una asignación incompatible puede producir desplazamientos innecesarios, esperas o redistribución de carga. La magnitud económica de esos efectos deberá medirse antes de calcular ahorros atribuibles al proyecto.

Entre las causas por contrastar están la dispersión de información sobre horarios, la ausencia de una verificación uniforme de especialidades y la falta de actualización de dependencias. Una actividad puede ser urgente y, aun así, no estar lista para empezar. La revisión de Gómez Sánchez et al. (2023) muestra que la programación de varios proyectos con recursos compartidos combina restricciones de recursos y precedencias. Esta relación permite delimitar el caso: una persona no puede atender dos actividades incompatibles, y una tarea sucesora necesita respetar sus trabajos previos. No se presupone que una regla de prioridad produzca por sí sola el mejor calendario global.

1. Formulación del problema

Continuación

Los datos principales son obras, fechas comprometidas, actividades, duración estimada, personal requerido, especialidades, jornadas, ausencias y asignaciones. También se requieren motivos de cambio, tiempos de traslado, avances validados e incidencias. Los procesos relevantes ocurren al preparar el plan semanal, confirmar una asignación, comunicar una reprogramación y comprobar el cierre. Hendrickson (2008) explica que las restricciones de recursos condicionan la programación y que la revisión interactiva del calendario puede ser necesaria. En este caso, la duración y los traslados se registrarán como estimaciones del supervisor, cuya calidad deberá contrastarse con la ejecución.

Las variables cuantitativas incluyen obras activas, horas disponibles y asignadas por trabajador, conflictos detectados, actividades sin responsable, minutos de planificación y cumplimiento semanal. Las variables cualitativas comprenden especialidad, prioridad, estado, zona y motivo de bloqueo. Se trabajará con intervalos de inicio y fin en la zona America/Guayaquil. Las jornadas disponibles y los traslados restringen qué secuencias resultan viables. La evaluación propuesta contempla observar cuatro semanas antes del piloto y cuatro durante su uso. Una reducción de conflictos deberá compararse con la carga de trabajo de cada período para evitar atribuir mejoras a una menor cantidad de obras.

El proyecto académico conserva Java Swing y PostgreSQL como restricciones tecnológicas del documento base. Los volúmenes de 100 obras, 200 trabajadores, 10 000 actividades y 20 usuarios simultáneos son datos de prueba propuestos, no cifras verificadas de la empresa. La simultaneidad agrega una dificultad: dos supervisores podrían evaluar la misma disponibilidad y confirmar decisiones incompatibles. La documentación de PostgreSQL Global Development Group (s. f.) describe bloqueos de filas que permiten coordinar transacciones concurrentes. Por tanto, validar una agenda visible en pantalla no bastará para garantizar integridad en el momento de confirmar.

La dimensión social se relaciona con claridad de horarios, carga de trabajo y adaptación de los usuarios al registro diario. La dimensión económica aparece en esperas y reprocesos potenciales. Los desplazamientos podrían tener un efecto ambiental, pero no se cuantificará una reducción de emisiones sin distancias y mediciones. Tampoco se atribuyen características culturales no documentadas al personal. El problema queda formulado como la dificultad de coordinar actividades y trabajadores entre obras bajo restricciones de disponibilidad, especialidad, precedencia y traslado, manteniendo información actualizada y responsabilidades verificables.

2. Alcance y módulos funcionales

La primera versión conserva los cinco módulos del Word. Incluye búsqueda, filtrado, priorización, detección de ciclos, validación temporal y cálculo de indicadores. Quedan fuera nómina, compras, facturación, rutas óptimas, aplicación móvil y optimización global automática. Las reglas no funcionales originales siguen vigentes; su verificación se planifica para etapas posteriores.

M1 · Obras

Propósito: Identificar frentes y compromisos. Actor: Administrador y supervisor.

Datos: Código, nombre, servicio, dirección, contacto, fechas, estado y versión.

Crear obra con código único (RF-01).

Filtrar por estado, servicio, zona y fechas (RF-02).

Modificar fechas con motivo y versión anterior (RF-03).

Dar de baja y exigir resolver asignaciones vigentes (RF-04).

Ordenar por entrega e identificar tareas vencidas o sin responsable (RF-05).

Reglas: Fin no anterior al inicio. Una obra terminada necesita reapertura autorizada. Baja lógica, sin borrar historial.

Relación: M3 aporta actividades; M4 aporta plan y responsables; M5 aporta avance.

Responsable preliminar: Daniel Valarezo.

2. Módulos funcionales

M2 · Trabajadores y disponibilidad

Propósito: Mantener capacidad de trabajo verificable. Actor: Administrador y supervisor.

Datos: Código, contacto, especialidades, estado, jornadas, bloqueos y cuenta de usuario.

Registrar trabajador y cuenta con rol (RF-06).

Buscar por especialidad, estado y fecha disponible (RF-07).

Actualizar datos y validar solicitudes de disponibilidad (RF-08).

Desactivar y resolver asignaciones futuras (RF-09).

Calcular horas libres/asignadas y filtrar candidatos compatibles (RF-10).

Reglas: Solo especialidades validadas. Un bloqueo no se aprueba sin resolver cruces. Se rechazan horarios de duración nula o negativa.

Relación: M4 consume la disponibilidad; M5 recoge novedades; seguridad controla los roles.

Responsable preliminar: Esteban Figueroa.

M3 · Actividades y prioridades

Propósito: Descomponer obras y definir un orden ejecutable. Actor: Supervisor.

Datos: Código, obra, descripción, duración, especialidad, cantidad de personas, fecha, prioridad, estado y dependencias.

Crear tarea de duración positiva en obra activa (RF-11).

Filtrar por obra, estado, prioridad o fecha (RF-12).

Modificar duración, prioridad y dependencias con revisión de asignaciones (RF-13).

Cancelar y liberar reservas futuras (RF-14).

Priorizar por alta–media–baja, fecha y código; detectar ciclos (RF-15).

Reglas: No se aceptan ciclos ni dependencia de una tarea consigo misma. Para esta versión se proponen dependencias dentro de la misma obra. Iniciar exige predecesoras terminadas.

Relación: M1 contiene las tareas; M4 consulta precedencias y prioridad; M5 valida terminaciones.

Responsable preliminar: Julián Jiménez.

2. Módulos funcionales

M4 · Planificación y asignación

Propósito: Asignar personal y horarios sin incompatibilidades. Actor: Supervisor.

Datos: Actividad, conjunto de trabajadores, inicio, fin, traslados, estado y versión del plan.

Confirmar validando especialidad, cantidad, disponibilidad y secuencia (RF-16).

Consultar agenda diaria/semanal por persona y obra (RF-17).

Reprogramar con motivo, revalidación y actualización de agendas (RF-18).

Cancelar reserva futura conservando historia (RF-19).

Detectar cruce, identificar reserva conflictiva y sugerir personas libres (RF-20).

Reglas: Solo supervisor confirma. Confirmación de todos los trabajadores en una transacción: todo o nada. Si faltan candidatos, la actividad queda pendiente. Traslados explícitos entre obras.

Relación: Integra M1–M3 y reacciona a bloqueos/avances de M5. Esteban apoya integridad de agenda.

Responsable preliminar: Cuarto integrante por definir.

M5 · Seguimiento e incidencias

Propósito: Distinguir avance informado de avance validado. Actor: Trabajador y supervisor.

Datos: Reporte, actividad, autor, fecha, porcentaje, evidencia opcional, incidencia, revisión y versiones.

Registrar avance entre 0 y 100 e incidencias (RF-21).

Consultar historial y filtrar incidencias (RF-22).

Corregir con motivo y validar, rechazar o pedir aclaración (RF-23).

Anular sin borrar evidencia (RF-24).

Calcular cumplimiento, señalar vencimientos y validar cierre (RF-25).

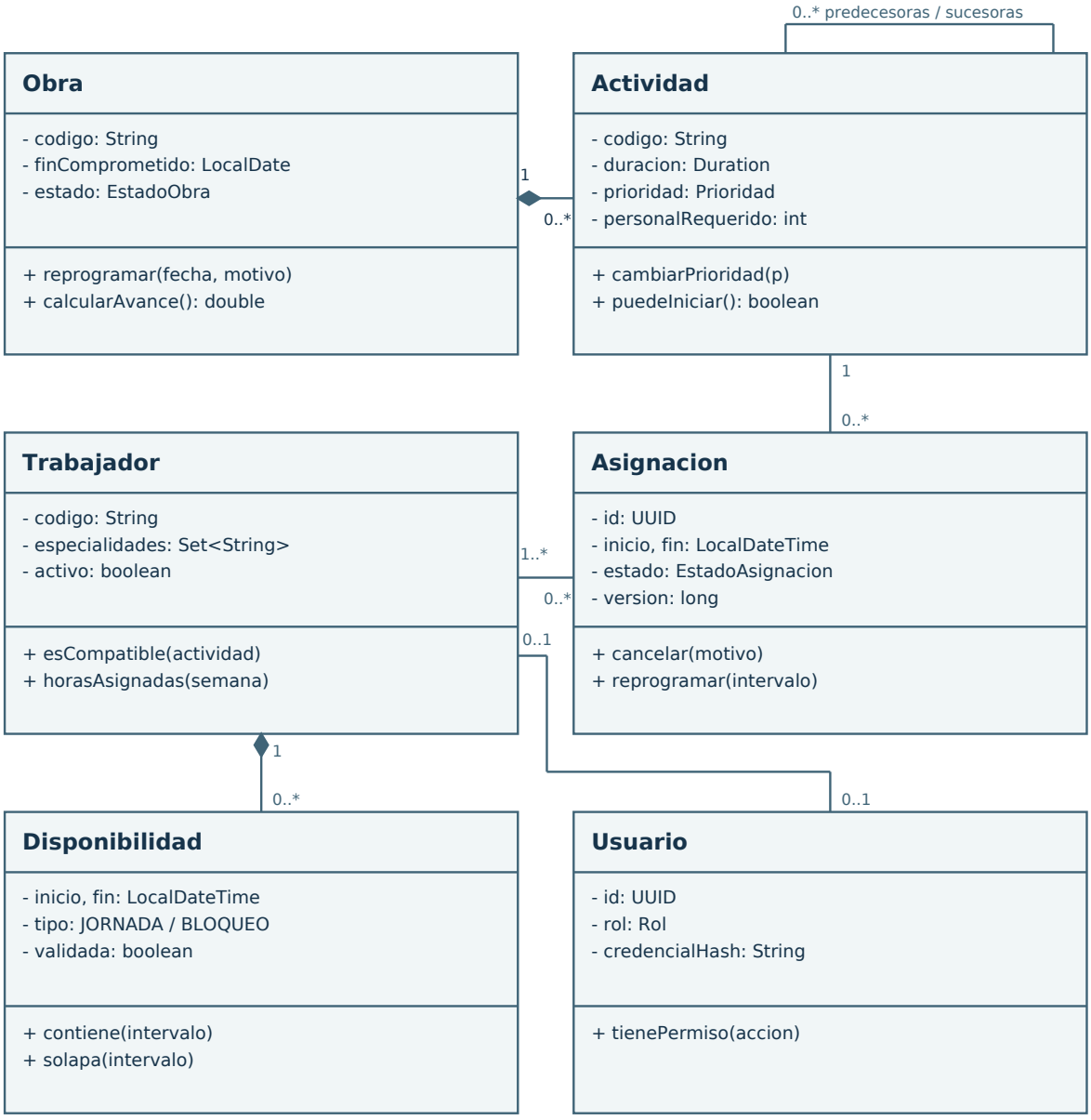
Reglas: Trabajador reporta solo tareas asignadas. Cierre requiere 100 % validado y ninguna incidencia abierta. El avance de obra pondera duración vigente y excluye canceladas.

Relación: M3 recibe estado de ejecución; M4 recibe novedades; M1 muestra indicadores.

Responsable preliminar: Daniel Valarezo; apoyo de Julián Jiménez.

3. Modelo orientado a objetos

Se propone separar dominio, servicios de aplicación, algoritmos, persistencia e interfaz Swing. Las clases de dominio contienen reglas locales; PlanificadorService coordina reglas entre objetos. Usuario usa un atributo rol, sin una jerarquía artificial de administradores, supervisores y trabajadores. La composición Obra–Actividad representa pertenencia exclusiva, no borrado físico en cascada.



Actividad depende de Actividad: 0..* predecesoras y 0..* sucesoras. Sin ciclos. Cada asignación pertenece a una actividad. Un trabajador puede participar en varias asignaciones sin solaparse.

Vista 1. Clases principales. Atributos y métodos representativos.

3. Relaciones y seguimiento



PlanSemanal referencia 0..* actividades comprometidas y 0..* asignaciones. Cada una puede aparecer en 0..* versiones. ReporteAvance tiene 0..1 revisor Usuario. Incidencia tiene 1 autor Usuario. Las versiones publicadas conservan el compromiso original.

Vista 2. Seguimiento y versiones del plan. Usuario y Actividad son las mismas clases de la vista 1.

3. Multiplicidades y reglas del UML

Relación	Multiplicidad y significado
Obra–Actividad	1 obra contiene 0..* actividades. Cada actividad pertenece a 1 obra. Composición, conservando baja lógica.
Actividad–Actividad	0..* predecesoras y 0..* sucesoras. Grafo dirigido acíclico, dentro de una obra.
Actividad–Asignacion	1 actividad tiene 0..* asignaciones. Cada asignación corresponde a 1 actividad.
Asignacion–Trabajador	Cada asignación confirma 1..* trabajadores, exactamente la cantidad requerida. Cada trabajador tiene 0..* asignaciones.
Trabajador–Disponibilidad	1 trabajador compone 0..* intervalos. Cada intervalo pertenece a 1 trabajador.
Trabajador–Usuario	0..1 cuenta por trabajador y 0..1 trabajador por cuenta. Un supervisor puede tener cuenta sin ser personal asignable.
Actividad–ReporteAvance / Incidencia	1 actividad recibe 0..* reportes y 0..* incidencias. Cada reporte/incidencia referencia 1 actividad.
Usuario–ReporteAvance	1 autor por reporte y 0..1 revisor. Un usuario puede autorizar o revisar 0..* reportes.
Usuario–Incidencia / EventoAuditoria	1 autor por incidencia o evento. Un usuario registra 0..* incidencias/eventos.
PlanSemanal–Actividad / Asignacion	Relaciones 0..* a 0..*. Las versiones publicadas guardan instantáneas del compromiso y del calendario, sin duplicar identidades de dominio.

PlanificadorService expondrá sugerirCandidatos(), validarAsignacion() y confirmarAsignacion(); GrafoDependencias, detectarCiclo() y ordenar(); AgendaTrabajador, buscarConflicto(). Son servicios/estructuras, no nuevas entidades del negocio.

Revisión requerida: abrir docs/uml/dizeo.drawio en diagrams.net, revisar ambas vistas y registrar fecha, participantes y cambios en docs/uml/REVISION.md. Las vistas del informe son una representación preliminar; la tabla hace explícitas todas las multiplicidades. No se declara una revisión del equipo que todavía no ha ocurrido.

4. Alternativas técnicas

Componente comparado: priorización de tareas, control de dependencias y detección de conflictos de agenda. Ambas alternativas aplican exactamente las mismas reglas de negocio. A será referencia de corrección; B será la estructura principal propuesta. Se comparan algoritmos en memoria, sin incluir latencia de red, consultas SQL ni esperas de bloqueo.

Notación: V = actividades, E = dependencias, W = trabajadores, A = reservas activas totales, k = intervalos de un trabajador, q = tareas elegibles, c = candidatos por especialidad, r = reservas devueltas. Las agendas incluyen asignaciones y bloqueos aprobados; las jornadas disponibles se normalizan por día. S = relaciones trabajador–especialidad.

Alternativa	Estructuras y algoritmos	Ventajas y límites
A · Listas y matriz	ArrayList para entidades y agenda; matriz booleana V × V para precedencias; búsqueda lineal; ordenamiento por comparador; DFS recorriendo filas de la matriz.	Implementación directa y fácil de inspeccionar. Explora toda la agenda y reserva memoria cuadrática para dependencias, incluso si son escasas.
B · Índices y grafo disperso	HashMap por ID; índice por especialidad; TreeMap por trabajador; PriorityQueue; grafo de listas de adyacencia; Kahn y consultas de vecinos.	Mejora consultas repetidas y dependencias dispersas. Exige mantener índices, reconstruir prioridades y preservar agendas sin cruces. No ofrece una solución óptima global.

Operaciones de agenda e identificación

Operación	A: tiempo / espacio adicional	B: tiempo / espacio adicional
Buscar trabajador por ID	O(W) / O(1)	O(1) esperado; O(W) conservador en peor caso / O(1)
Validar cruce de un trabajador	O(k) / O(1)	O(log k) / O(1), bajo invariante de intervalos no solapados
Insertar reserva validada	O(1) amortizado al final / O(1)*	O(log k) / O(1)*
Listar r reservas de un rango	O(k + r) / O(r)	O(log k + r) / O(r)
Filtrar candidatos compatibles y libres	O(W + A) / O(c), recorriendo agendas de candidatos	O(c log k máximo) esperado / O(c), con índice por especialidad

* Espacio incremental lógico por reserva. Una expansión interna de ArrayList puede requerir O(k) temporal. El almacenamiento persistente de agendas es O(W + A). Fuentes: Oracle (s. f.-a, s. f.-c). Los costos compuestos se derivan de las operaciones del diseño.

4. Prioridades y dependencias

Operación	Alternativa A	Alternativa B
Construir dependencias	$O(V^2 + E)$ tiempo y $O(V^2)$ memoria, incluida inicialización de matriz.	$O(V + E)$ tiempo y memoria mediante listas de adyacencia.
Detectar ciclo u obtener orden topológico	DFS con matriz: $O(V^2)$ tiempo; $O(V)$ auxiliar además de matriz.	Kahn: $O(V + E)$ tiempo; $O(V)$ auxiliar además del grafo.
Elegir y retirar próxima tarea de q elegibles	Recorrido para mínimo y borrado: $O(q)$ tiempo; $O(1)$ auxiliar.	poll() en PriorityQueue: $O(\log q)$ tiempo; $O(1)$ auxiliar, cola $O(q)$.
Mostrar todas las tareas en orden	Ordenar copia: $O(q \log q)$ tiempo; $O(q)$ auxiliar.	Copiar y extraer de cola: $O(q \log q)$ tiempo; $O(q)$ auxiliar. Iterar la cola NO ordena.
Cambiar prioridad arbitraria	Encontrar y actualizar en lista: $O(q)$. Se vuelve a ordenar al consultar.	remove(objeto) + offer: $O(q)$. No mutar claves dentro del heap. Alternativa simple: reconstruir por inserciones $O(q \log q)$.
Memoria total relevante	$O(W + A + S + V^2)$, sin contar reportes ni historial.	$O(W + A + S + V + E)$, sin contar reportes ni historial.

Supuestos y alcance de las cotas

En B, TreeMap ordena reservas por inicio. Como los intervalos confirmados no se solapan, basta revisar el vecino anterior y el siguiente para detectar una incompatibilidad con una nueva reserva. Se comprueban además los traslados desde la obra anterior y hacia la siguiente. Si la agenda importada contiene cruces, primero se rechaza o corrige: sin ese invariante, la consulta de vecinos no es suficiente. Los bloqueos aprobados usan el mismo índice; las jornadas se validan por separado.

Dos intervalos $[a,b)$ y $[c,d)$ se cruzan cuando $a < d$ y $c < b$. Si $b = c$, solo son compatibles cuando el traslado requerido es cero. La consulta por rango incorpora el intervalo anterior al inicio del rango si continúa dentro de él. Las dependencias cambian con validación completa $O(V + E)$, una decisión deliberada de sencillez.

Kahn usa ArrayDeque para detectar ciclos. La cola de prioridad contiene únicamente tareas elegibles para la operación elegida. Para planificar, sus predecesoras deben tener una secuencia prevista compatible; para iniciar ejecución deben estar terminadas. Kahn con selección por prioridad en cada extracción costaría $O(E + V \log V)$, en lugar de $O(V + E)$. Fuente de operaciones del heap: Oracle (s. f.-b).

5. Selección y fundamentación

Se selecciona la alternativa B porque la operación central requiere consultar repetidamente disponibilidad, precedencias y prioridades sobre datos que cambian. A conserva valor como modelo de referencia para contrastar resultados con conjuntos pequeños. La elección es una hipótesis técnica sustentada en complejidad; todavía no existen mediciones del sistema implementado.

Criterio	Fundamentación de B
Adecuación	Representa por separado identidad, tiempo y precedencia. Conserva aprobación del supervisor y explicaciones de conflictos.
Tiempo	Revisar un intervalo pasa de O(k) a O(log k); recorrer dependencias dispersas pasa de $O(V^2)$ a O(V + E). Las consultas generales y los KPI siguen necesitando recorrer datos relevantes.
Memoria	Con V = 10 000 y E = 30 000, A representa 100 millones de celdas lógicas; B representa 10 000 vértices y 30 000 aristas. Son conteos teóricos, no bytes medidos de Java.
Escalabilidad	Los índices favorecen consultas frecuentes. Si E se aproxima a V^2, el grafo pierde la ventaja asintótica de memoria. Un heap no acelera listar todo el orden.
Mantenibilidad	Servicios encapsulan índices; PostgreSQL conserva la fuente de verdad. Reconstrucción tras cambios confirmados evita estados divergentes.
Dificultad	Usar colecciones estándar reduce código de infraestructura. El equipo implementará comparadores, Kahn y validación de intervalos, sin programar un árbol rojo-negro desde cero.

Política de decisión propuesta

El supervisor define el intervalo y solicita sugerencias. El sistema filtra especialidad y disponibilidad, ordena candidatos por menor carga semanal y desempata por código, y permite elegir el equipo completo. Ordenar c candidatos agrega O(c log c), más el cálculo de sus cargas. Esta heurística favorece una distribución transparente, pero no garantiza minimizar retrasos ni equilibrar globalmente todas las obras. Si no se completa el personal requerido, no se confirma una reserva parcial.

Confirmación consistente

La confirmación se propone en un servicio Java central que atiende Swing y persiste en PostgreSQL. Se bloquean las filas de trabajadores en orden de código, se relee su agenda, se revalidan condiciones y se guardan asignación, historial y versión en una transacción. La cancelación y reprogramación usan el mismo protocolo. Los índices en memoria se actualizan después del commit. Es un refinamiento arquitectónico propuesto para satisfacer la validación del lado servidor del Word, pendiente de aprobación del equipo (PostgreSQL Global Development Group, s. f.).

6. Plan preliminar de implementación

Responsable	Componentes	Evidencia esperada
Daniel Valarezo	M1 Obras y M5 Seguimiento; coordinación de documentación.	Filtros/orden de obras, avance ponderado, cierre validado y trazabilidad.
Esteban Figueroa	M2 Trabajadores; apoyo a agenda y persistencia.	Compatibilidad, jornadas, bloqueos y pruebas de reservas concurrentes.
Julián Jiménez	M3 Actividades; grafo y prioridades.	Kahn, comparador total, casos de ciclo y dependencias.
Cuarto integrante por definir	M4 Planificación; coordinación de servicio central e integración Swing.	Sugerencias, calendario y confirmación transaccional con apoyo de Esteban.

Asignación preliminar, no evidencia de aportes ya realizados. Si se incorpora un quinto integrante, puede asumir integración/persistencia y pruebas. La estructura se plantea para cuatro integrantes sin inventar nombres.

Hitos relativos hacia Progreso 2

Hito	Trabajo y criterio de salida
Semana 1	Validar problema y alcance con Dizeo; completar equipo; revisar UML y contratos. Registrar observaciones docentes como requisitos de mejora.
Semana 2	Implementar dominio, estructuras A de referencia y B; verificar mismas respuestas en datos pequeños, ciclos, empates y límites horarios.
Semana 3	Integrar servicio, PostgreSQL y Swing; asegurar transacciones, permisos, auditoría y actualización de agendas.
Semana 4	Pruebas integradas, medición del percentil 95, casos de concurrencia, documentación y ensayo grupal. Fechas reales se ajustarán al calendario del curso.

Estructura del repositorio inicial

README.md en la raíz; docs/informe para el informe PDF; docs/presentacion para la presentación técnica; docs/uml para el diagrama editable y su acta de revisión; docs/plan para conservar las observaciones del docente.

Se entrega un repositorio Git local con commit inicial y README, limitado a análisis y diseño. No incluye código Java, prototipo ni configuración ejecutable. El README indica cómo publicar el repositorio en GitHub cuando el equipo defina su cuenta y URL.

6. Riesgos y validación prevista

Riesgo	Mitigación concreta	Comprobación
Doble reserva simultánea	Transacción central con bloqueo por trabajador y relectura; orden fijo de adquisición.	50 pares de intentos incompatibles: como máximo uno confirma en cada par.
Índices desactualizados	Una fuente persistente; actualizar tras commit y reconstruir al recuperar sesión.	Comparar agenda reconstruida con registros vigentes tras cancelar/reprogramar.
Ciclos o secuencia incorrecta	Kahn antes de aceptar dependencias y validación de fin de predecesoras al planificar.	A→B→C→A se rechaza; tarea no inicia con predecesora incompleta.
Traslados omitidos	Validar ambos vecinos y exigir duración de traslado registrada.	Caso T01: A 08:00–12:00; B 12:00 rechazada; B 12:30 admisible con traslado de 30 min.
Prioridad obsoleta en heap	Retirar/reinsertar o reconstruir al cambiar criterio.	Empate resuelto por fecha y código; actualización cambia el próximo elemento.
Alcance excesivo y equipo incompleto	Cerrar responsables, priorizar flujo completo y conservar trazabilidad de RF pendientes.	Revisión semanal con responsable y condición de aceptación por módulo.
Datos insuficientes o sensibles	Usar datos sintéticos; entrevistas con evidencia mínima y contactos restringidos por rol.	No introducir datos reales en Git ni inventar resultados del piloto.
Pérdida de observaciones docentes	Registrar texto, fecha, requisito afectado, responsable y evidencia de corrección.	Cada observación tiene estado y cierre revisado para Progreso 2.

Criterios de aceptación heredados y pruebas

En 2 núcleos/4 GB, con 100 obras, 200 trabajadores, 10 000 actividades y 20 usuarios simultáneos, medir 100 solicitudes por operación: consultas con p95 ≤ 2 s y conflictos con p95 ≤ 3 s. Registrar tiempos, tamaño de E, distribución de agenda y consumo de memoria. No convertir O(log k) en una promesa de milisegundos. Verificar además roles, credenciales protegidas, respaldo/restauración, auditoría y zona horaria conforme al Word.

Cumplimiento semanal = tareas del plan publicado terminadas a tiempo / tareas comprometidas × 100. Mantener en el denominador las canceladas después de publicar. Sin denominador, mostrar “No aplica”. Meta inicial del piloto: 90 %, sujeta a validación. Una agenda sin conflictos no demuestra cumplimiento de plazos.

Declaración de uso de IA

Herramienta	Uso realizado	Resultado incorporado	Forma de verificación
ChatGPT	Lectura del documento base, organización del avance y apoyo al razonamiento técnico.	Borrador del informe, propuesta UML editable, comparación de alternativas, diapositivas y esqueleto de repositorio.	Revisión del equipo pendiente: contrastar RF, abrir UML, comprobar cotas con ejemplos y explicar decisiones sin depender de IA.
ChatGPT con consulta web	Localización y consulta de fuentes editoriales, universitarias y documentación oficial.	Referencias y sustento de restricciones/estructuras.	Se contrastaron metadatos y documentación consultada. El equipo debe comprobar enlaces y correspondencia de citas antes de entregar.

Este documento declara el apoyo recibido y no atribuye al equipo pruebas, entrevistas, implementación o revisiones aún no realizadas. La responsabilidad académica y técnica corresponde a sus integrantes. Se conserva la distinción entre evidencia del Word, supuestos de diseño y metas futuras.

Presentación técnica: 9 minutos y 30 segundos

Diapositivas	Contenido	Expositor	Tiempo
1–2	Proyecto, problema y evidencia	Daniel	1 min 25 s
3–4	Variables, restricciones y módulos	Esteban	1 min 40 s
5–6	UML y alternativas	Julián	2 min 15 s
7–8	Comparación y selección	Cuarto integrante	2 min
9	Caso de asignación	Esteban	1 min
10	Plan y cierre	Daniel	1 min 10 s

Todos participan. La presentación incluye notas de exposición y referencias por diapositiva. El tiempo excluye preguntas. El cuarto integrante deberá reemplazarse por su nombre real.

Referencias

Durdyev, S., & Hosseini, M. R. (2020). Causes of delays on construction projects: A comprehensive list. *International Journal of Managing Projects in Business, 13*(1), 20–46. https://doi.org/10.1108/IJMPB-09-2018-0178

Gómez Sánchez, M., Lalla-Ruiz, E., Fernández Gil, A., Castro, C., & Voß, S. (2023). Resource-constrained multi-project scheduling problem: A survey. *European Journal of Operational Research, 309*(3), 958–976. https://doi.org/10.1016/j.ejor.2022.09.033

Hendrickson, C. (2008). *Project management for construction: Fundamental concepts for owners, engineers, architects and builders* (versión 2.2, cap. 11). Carnegie Mellon University. https://www.cmu.edu/cee/projects/PMbook/11_Advanced_Scheduling_Techniques.html

Oracle. (s. f.-a). *HashMap (Java SE 21 & JDK 21).* https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html

Oracle. (s. f.-b). *PriorityQueue (Java SE 21 & JDK 21).* https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html

Oracle. (s. f.-c). *TreeMap (Java SE 21 & JDK 21).* https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeMap.html

PostgreSQL Global Development Group. (s. f.). *Explicit locking.* PostgreSQL 18 documentation. https://www.postgresql.org/docs/18/explicit-locking.html

Valarezo, D., Figueroa, E., & Jiménez, J. (2026). *Levantamiento de requerimientos para Dizeo: Sistema de planificación y asignación de trabajadores y actividades de obra* [Trabajo académico no publicado]. Universidad de las Américas.

Nota bibliográfica: el Word cita Durdyev y Hosseini con 2019, año de publicación anticipada. Aquí se usa 2020, año del volumen 13(1) indicado por la editorial. La pieza comercial de Dizeo se conoce a través del documento base; no se presenta como una fuente externa consultada directamente.